



OptiGrid

Energy Intelligence Platform

Testing Policy Document

Version 1.0

July 30, 2026

Team Coreflow

COS 301 - Capstone Project

University of Pretoria

Contents

Document Revision History	2
1 Introduction	3
1.1 Purpose	3
1.2 Scope	3
2 Testing Objectives	3
3 Types of Testing	3
3.1 Unit Testing	3
3.2 Integration Testing	3
3.3 End-to-End Testing	4
4 Test Coverage	4
5 Tools and Environments	4
5.1 Testing Frameworks and Libraries	4
5.2 Test Environments	4
6 Running the Tests	5
7 Continuous Integration	5
8 Defect Management	5
9 Definition of Done	6
10 Roles and Responsibilities	6

Document Revision History

Version	Date	Description of Change	Author
1.0	July 30, 2026	Initial version.	Team Coreflow

1 Introduction

1.1 Purpose

This document outlines software testing standards in the OptiGrid Platform. It defines the procedures and responsibilities of testing the product to deliver and maintain a high quality software.

1.2 Scope

The policy covers the Next.js frontend together with Express core API at the backend. The ingestion and analytics service implemented using python are covered by testing as well. So, it applies to all code that get merged into the `develop` and `main` branches.

2 Testing Objectives

- Confirm that each feature behaves as specified before it is merged and deployed.
- Verify that role based access control is enforced for all endpoints to protect against attacks.
- Identify and prevent regressions early in the pipeline before costly crashes occur.
- Keep the main branches in a working state and functional at all times.

3 Types of Testing

The project has three layers of testing. And every level test a different class of errors and defects. For a feature to be considered functional, it has to pass all levels of testing.

3.1 Unit Testing

Unit tests consider a single function, component, or a service on its own. Any other instance or service needed by the unit is mocked to produce certain behaviours.

- **Frontend:** React components and helper modules. Use Jest with React Testing Library. Network calls are mocked at `fetch`.
- **Backend:** controllers, services, validation schemas, and middleware. Use Jest. Prisma and Supabase are mocked.
- **Python services:** ingestion and analytics logic. Use pytest.

3.2 Integration Testing

Integration tests check the API from end to end at the HTTP layer. It uses the real database rather than mocks. A disposable PostgreSQL container is started with Testcontainers, the Prisma schema is applied to it. The requests are issued with supertest. The test verifies authentication and role permissions. It also verifies the database behaviour and its unique identifiers and so on.

3.3 End-to-End Testing

End to end tests runs the real application in a browser using Playwright, against the frontend alongside the core API and a local Supabase instance. They cover the main user use cases such as signing up, logging in, and managing buildings and confirm they are working as expected.

4 Test Coverage

The coverage is reported by Jest when the backend suite gets run with the `-coverage` flag. There is no numeric coverage threshold enforced currently. Instead of that, it is required that every new feature is merged with tests at the levels relevant to it, as described in Section 9.

Table 1: Test suites by level

Level	Files	Area covered
Unit (frontend)	35	Pages, components, and API proxy routes
Unit (backend)	28	Controllers, services, validation, middleware
Unit (Python)	4	Ingestion and analytics logic
Integration	17	Authentication, buildings, sensors, telemetry, analytics, preferences
End-to-end	10	Authentication, buildings, settings, live readings

5 Tools and Environments

5.1 Testing Frameworks and Libraries

Table 2: Testing tools used in the project

Tool	Version	Purpose
Jest	29.7	Unit and integration test runner
React Testing Library	16.2	Rendering and querying React components
supertest	7.1	Issuing HTTP requests to the Express app
Testcontainers	10.2	Disposable PostgreSQL for integration tests
Playwright	1.52	Browser driven end-to-end tests
pytest	8–9	Python service tests

5.2 Test Environments

Unit tests do not require external services. Integration tests on the other hand require Docker in order to start the PostgreSQL container and the Supabase CLI for local authentication. End-to-end tests require the Playwright browser binaries in addition to that. For the integration tests, the database is reset between tests, so suites do not depend on one another or on their execution order.

6 Running the Tests

Integration tests are run from `backend/core` with `pnpm test:integration`. The developers are expected to run the unit tests before pushing and the remaining levels are run by the pipeline.

Table 3: Commands, run from the repository root

Command	Runs
<code>pnpm test:node</code>	Frontend and backend unit tests
<code>pnpm test:python</code>	Ingestion and analytics unit tests
<code>pnpm test:all</code>	All unit tests, Node and Python
<code>pnpm test:e2e</code>	Playwright end-to-end tests

7 Continuous Integration

Every push and pull request targeting `develop` or `main` triggers the GitHub Actions pipeline, which runs the following jobs:

- **Frontend:** lint, unit tests, then build.
- **Backend core:** lint, unit tests, and build against a real PostgreSQL service and a local Supabase instance.
- **Python services:** pytest for ingestion and analytics.
- **Integration:** Terraform validation and the authentication suite.
- **End-to-end:** Playwright against a local Supabase instance.

8 Defect Management

Defects are tracked as GitHub issues. A branch is then created to fix the area affected. The workflow goes as follows: reproduce the defect, add a failing test that captures the defect, fix the code until that test passes, then confirm the wider suite still passes. The regression test written to catch the bug gets committed with the fix so the same defect cannot return again unnoticed.

Table 4: Defect severity levels

Severity	Description	Response
Critical	Blocks a core user journey, or exposes and leak data to the wrong users	Fixed immediately, before other work
Major	A feature is broken or gives incorrect results, but is relatively functional	Fixed in the current sprint
Minor	Cosmetic or low impact issue	Scheduled to fix in a later sprint

9 Definition of Done

A feature or fix is complete and considered "Done" when all of the following hold:

- Unit tests cover the new logic, including failure paths.
- Any additions or changes to API endpoints are covered by integration tests.
- The full test suite passes locally.
- All continuous integration jobs pass.
- The change has been reviewed and approved by 2 other team members minimally.

10 Roles and Responsibilities

Table 5: Testing roles and responsibilities

Role	Responsibilities
Developer	Writes tests for the feature, and runs the suite before pushing
Reviewer	Confirms the tests are present sufficient, and passing before approving
Team	Keeps the main branches green and make it a priority to fix a failing pipeline